hashlock.

2025

Penetration Testing

ForeGate (Prediction Market)

September 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

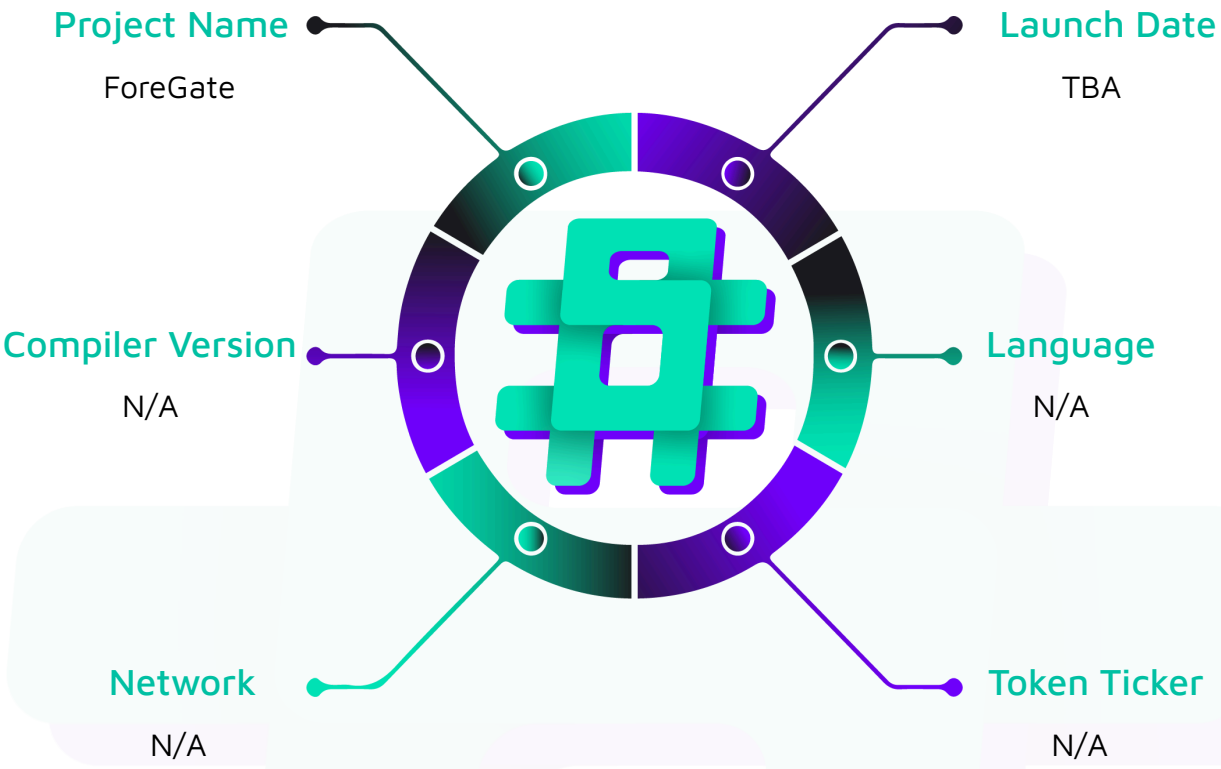
The ForeGate team partnered with Hashlock to conduct a security Penetration Test of their ForeGate Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

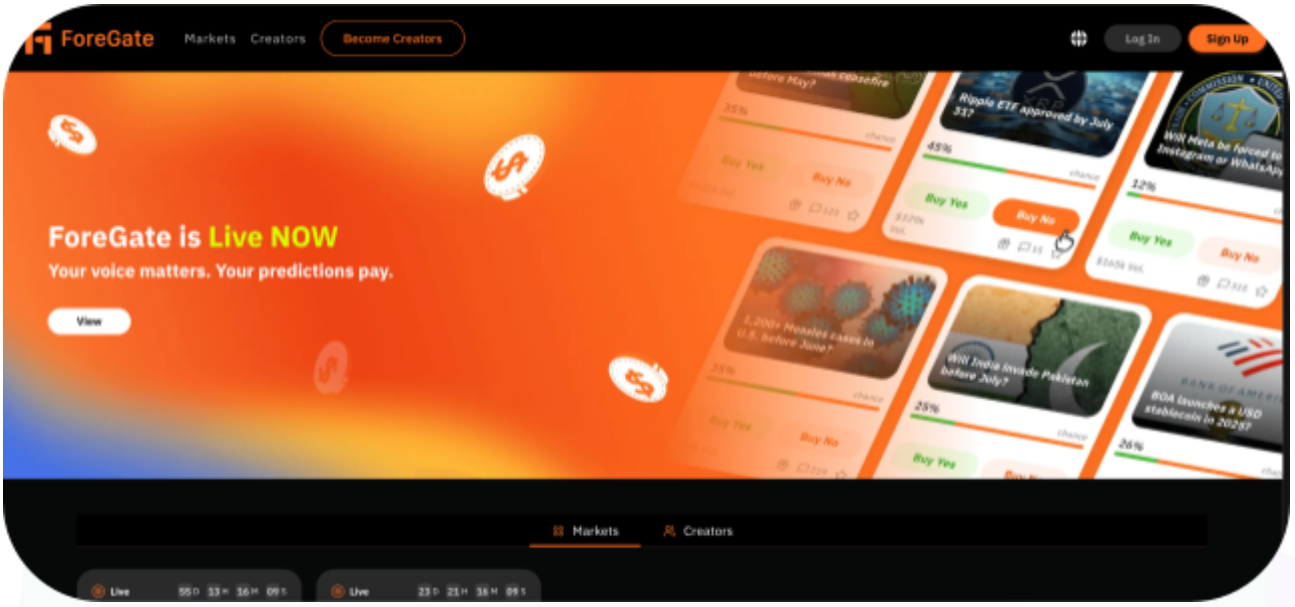
ForeGate is a next-generation prediction market platform, empowering users to forecast real-world events and trade on their outcomes. Whether you're a professional trader, industry expert, or just someone who follows trends, ForeGate offers a transparent, data-driven, and engaging marketplace where your insights can translate into real earnings.

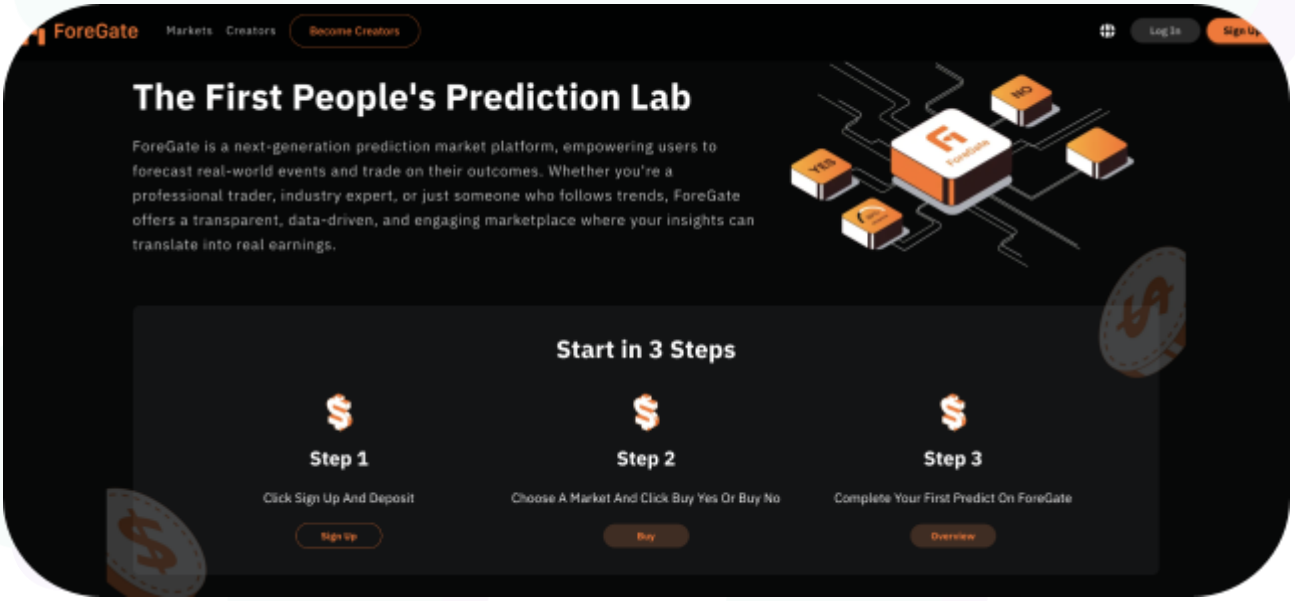
Project Name: ForeGate
Project Type: Prediction Market
Logo:

Visualised Context:



Project Visuals:





Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the ForeGate project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**ForeGate Project Code base**
Platform	**Ethereum / Solidity/Web apps & APIs**
Penetration Test Date	**September, 2025**
Repository/Website URL/IP Tested	https://github.com/gph-tech/signing-service-rust
GitHub Commit Hash	1ea257774d2ca7e070d8b4f54b526fa8a8a96194
Fix Review GitHub Commit Hash	19fa6eaba835c27a5c6a070d8ff90e588975808e

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 High-severity vulnerability

1 Medium-severity vulnerability

2 Low-severity vulnerabilities

1 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] src/routes.rs#authorize - Hardcoded token authentication and lack of access isolation

Description

The authentication model relies on a single hardcoded bearer token (`"catnboost"`) inside the `authorize` function. If this token is known to attackers (e.g., from brute force), they will gain full access to all privileged entry points.

Additionally, the `_auth_token` parameter that is passed to handlers is never validated against a specific `app_id` or `user_id`. This means there is no per-user or per-app isolation, allowing an authenticated user to access all other users' information.

Impact

The entire system's confidentiality and integrity will be compromised. For example, a malicious actor can retrieve other users' private keys via the `export_key` function, or abuse the `sign_transaction` function to steal user funds.

Recommendation

Implement proper per-app and per-user authentication mechanisms. Additionally, enforce authorization checks via the `auth_token` parameter so that only the owner can access their own wallet.

Status

Resolved

Medium

[M-01] src/handlers.rs#sign_transaction, new_wallet, export_key - Decrypted private key not zeroized

Description

Decrypted private keys in the `sign_transaction` (`user_decrypted_key` and `gas_decrypted_key` variables), `new_wallet`, and `export_key` functions are kept in memory after use. This is problematic because sensitive key material should be wiped immediately after constructing the `Keypair`.

Impact

Potential private key leakage risk if the memory is dumped or reused.

Recommendation

Use the zeroize crate to zeroize key buffers after use.

Status

Resolved

Low

[L-01] src/handlers.rs#new_app, new_wallet - Incorrect IDs and timestamps returned

Description

When inserting new rows, the `new_app` function returns `id: 0` and `created_at: 0` instead of the actual auto-increment ID and current Unix timestamp. Similarly, the `new_wallet` function also returns `id: 0` instead of the real DB ID.

Impact

API consumers may misinterpret the application or wallet state.

Recommendation

Fetch and return the actual row values after insert.

Status

Resolved

[L-02] src/crypto.rs#encrypt_message - Hardcoded default AES key

Description

The `encrypt_message` function uses a hardcoded fallback AES key if no key is provided (`let key_string = key_string.unwrap_or("a8GfZLp2Qw9VrD6Y");`). This logic is not required because `src/handlers.rs:245` always provides the key as `Some(key)`.

Impact

If future code paths allow providing `key_string` as `None`, the encryption mechanism would fall back to using the hardcoded key. This would render all encrypted messages trivially reversible to attackers.

Recommendation

Remove the hardcoded default and require the caller to provide keys at all times. Consider returning an error if no key is supplied.

Status

Resolved

QA

[Q-01] src/handlers.rs#new_app - Typographical errors in logs

Description

Log messages in line 391 use "`requet`" instead of "`request`".

Recommendation

Update the log to use "`request`".

Status

Resolved

Conclusion

After Hashlock's analysis, the ForeGate project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au